

# MPI - Collective Communication and Computation

- involves all processes in communicator
  - MPI\_COMM\_WORLD is default
  - can create own subsets
  - MPI-2+ can create even bigger sets if dynamic process allocation is supported
- programs using only collective communication are easy to understand
  - every process does roughly the same thing
  - no inventive communication patterns
- functions for collective communication are optimized
  - devised by experts
  - detailed implementation depends on infrastructure
    - existing protocols in network infrastructure (broadcast)
- all collective functions must be called by all processes in the communicator
- functions work with any number of processes from 1 onwards
- all collective functions are blocking (MPI-1, MPI-2)
- there are no tags
- basic data types (MPI-1, MPI-2)
- types of collectives
  - synchronization
  - data transfer
  - collective computation

## Synchronization

- MPI\_Barrier
  - rarely used
  - for performance measurements

## Data Transfer

- MPI\_Bcast
  - one to all (broadcast)

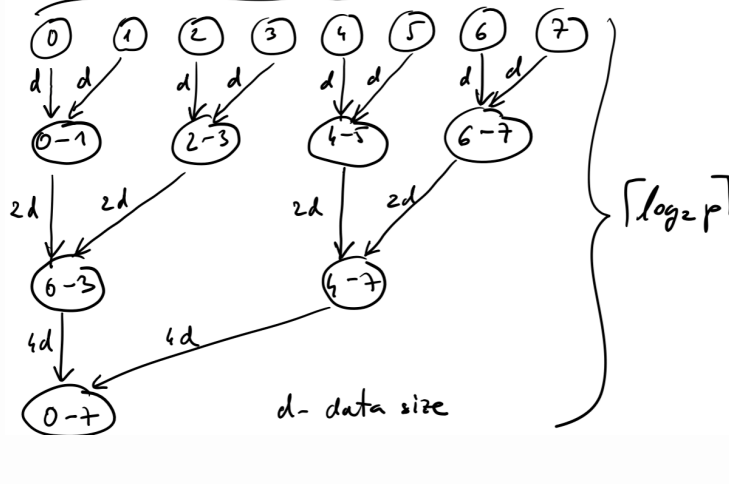


- MPI\_Scatter and MPI\_Gather

- scatters or gathers data across processes in the same communicator



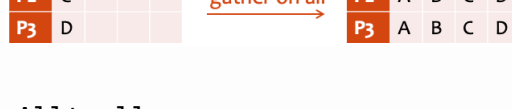
- expect all data chunks to be of the same size
- root process takes care of one data chunk
- some parameters are valid on side of sender, some on side of receiver
- tree-like implementation of gather



- MPI\_Scatterv and MPI\_Gatherv
  - more general but slower functions
  - size of data chunk can vary
- MPI\_Allgather

- combines gather and broadcast

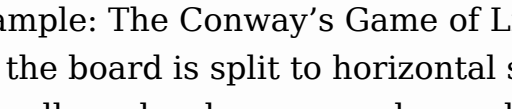
- can be efficiently implemented by only one pass of the tree



- MPI\_Alltoall

- transpose of data

- tricky to implement efficiently



- example: The Conway's Game of Life
  - the board is split to horizontal stripes
  - cells on borders are exchanged via MPI\_Sendrecv
  - code:
    - [conway.c](#)
    - [board.h](#)
    - [conway.sh](#)

## Collective computation

- MPI\_Reduce
  - reduces data from several processes
  - reduce operations
    - extreme: MPI\_MIN, MPI\_MAX
    - sum and product: MPI\_SUM, MPI\_PROD
    - logical operations: MPI\_LAND, MPI\_LOR, MPI\_LXOR
    - bit-wise operations: MPI\_BAND, MPI\_BOR, MPI\_BXOR
    - extreme with location: MPI\_MAXLOC, MPI\_MINLOC



- MPI\_Scan and MPI\_Exscan

- inclusive and exclusive scan

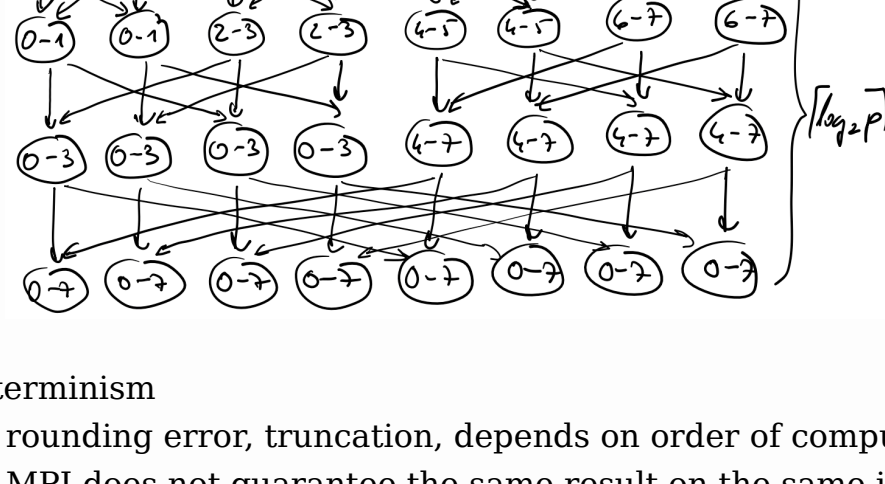


- MPI\_Allreduce

- combination of reduce and broadcast



- can be implemented with one pass of a tree



- determinism
  - rounding error, truncation, depends on order of computation
  - MPI does not guarantee the same result on the same input
    - encouraged but not required
    - not all applications need it
    - more efficient implementations of collectives are possible without it
- example: reduce with location information
  - combined data type
  - code:
    - [maxloc.c](#)
    - [maxloc.sh](#)

## Advanced MPI Features

- data types
- communicators
- virtual topology
  - reflects actual system configuration
  - Cartesian, graph
- MPI-IO
- collective functions
  - neighborhood
  - immediate